Malaviya National Institute of Technology Jaipur

Department of Artificial Intelligence and Data Engineering

Information Security — Course Code: 24AIT353

Session: 2025–2026

Course Instructor: Dr. Ramesh Babu Battula

Min-K% Probability Based Pretrained Code Detection

Statistical Membership Inference Analysis Using Perplexity and Min-K% Metrics

A Lightweight Security Analysis Framework Inspired by ICLR 2024 Research

Submitted by:

Name:	Pathuri Vivek Chowdary
Registration Number:	2023UAI1814
Department:	Artificial Intelligence and Data Engineering
Institute:	MNIT Jaipur

May 9, 2026

Abstract

This project studies, implements, and partially reproduces the membership inference framework introduced in the ICLR 2024 paper *Detecting Pretraining Data from Large Language Models* [1], applied specifically to the domain of source code. The paper proposes **Min-K% Prob**, a reference-free membership inference method that identifies whether a given text sample was part of a language model's pretraining corpus, relying solely on the minimum $k\%$ of token-level log-probabilities assigned by the target model.

This reproduction study applies the Min-K% Prob framework alongside classical perplexity-based baselines to a controlled code membership inference task using the `detect-pretrain-code` repository [2]. Experiments were conducted by comparing member and non-member code samples across eleven detection strategies, including raw perplexity (`ppl`), perplexity normalised by a reference model (`ppl/Ref_ppl`), lowercase-normalised perplexity (`ppl/lowercase_ppl`), zlib-compressed perplexity (`ppl/zlib`), and Min-K% Prob for $k \in \{5, 10, 20, 30, 40, 50, 60\}$.

The experimental results reveal a clear stratification in detection capability. Raw perplexity, zlib normalisation, and all Min-K% variants achieve **AUC = 1.000** and **Accuracy = 1.000**, demonstrating perfect statistical separability between member and non-member code. In contrast, reference model normalisation yields **AUC = 0.500** (random chance), while lowercase normalisation collapses to **AUC = 0.000**, inverting detection entirely. These findings confirm that LLMs memorise syntactic surface features of source code at very high fidelity, and that Min-K% Prob provides a robust, reference-free mechanism for detecting such memorisation. The project further discusses implications for data privacy, copyright protection, and the security risks posed by memorisation in large-scale code models.

Contents

Abstract **i**

1 Introduction **1**

2 Related Work **2**
2.1 Membership Inference Against Machine Learning Models 2
2.2 Perplexity-Based Detection 2
2.3 Min-K% Prob 2
2.4 Code-Specific Memorisation 2

3 Background Theory **3**
3.1 Language Model Probability and Perplexity 3
3.2 Normalised Perplexity Variants 3
3.3 Min-K% Prob 4
3.4 Evaluation Metrics 4

4 Threat Model and Security Context **4**
4.1 Adversary Capabilities 4
4.2 Attack Goals 5
4.3 Defender Perspective 5

5 Dataset and Repository Analysis **5**
5.1 Research Paper and Repository Overview 5
5.2 Repository Structure 6
5.3 Dataset Construction 6
5.4 Target Model 7

6 Implementation Details **7**
6.1 Environment Setup 7
6.2 Scoring Functions 7
6.3 Evaluation Pipeline 8

7 Experimental Setup **8**
7.1 Evaluation Protocol 8
7.2 Scoring Functions Evaluated 9

8 Results and Analysis **9**
8.1 Quantitative Results 9
8.2 ROC Curve Analysis 10
8.3 Analysis: Why Raw Perplexity Succeeds 10
8.4 Analysis: Why zlib Normalisation Also Succeeds 11

8.5 Analysis: Why Reference Model Normalisation Fails 11
8.6 Analysis: Why Lowercase Normalisation Inverts Detection (AUC = 0.000) 11
8.7 Analysis: Why All Min-K% Variants Achieve AUC = 1.000 12
8.8 Implications for LLM Memorisation 12

9 Pipeline Execution Summary **13**

10 Limitations **13**
10.1 Controlled Dataset Bias 13
10.2 Single Target Model 14
10.3 Min-K%++ Not Evaluated 14
10.4 No Adversarial Defences Evaluated 14

11 Conclusion **14**

List of Figures

1 Conceptual illustration of the membership inference attack pipeline. A query sample x is passed to the target LLM $\mathcal{M}$, which returns token-level log-probabilities. A scoring function (perplexity or Min-K% Prob) maps these probabilities to a scalar membership score, which is thresholded to classify x as a member or non-member of the training corpus. 3

2 ROC curves (log-log scale) for all eleven scoring functions on the code membership inference task. Nine of eleven methods achieve perfect AUC = 1.000 (blue, green, red, purple, brown, pink, grey, olive, and cyan lines collapsed at the top-left corner). The orange line (`ppl/Ref_ppl`) lies along the diagonal (AUC = 0.500, random chance). Curves for Min-K% Prob variants at all k values are indistinguishable from the perfect classifier line. 10

3 Conceptual visualisation of the per-token log-probability distributions for a member (top) and a non-member (bottom) code sample. Member tokens are concentrated near zero (high probability); non-member samples exhibit a heavier tail of very low-probability tokens, exactly the tokens captured by the Min-K% selection. 12

List of Tables

1 Dataset statistics used in the code membership inference experiments. . . 6

2 Summary of all eleven scoring functions evaluated in this study. 9

3 Membership inference results for all eleven scoring functions. AUC = Area Under ROC Curve; Acc = Balanced Accuracy; TPR = True Positive Rate at 0.1% FPR. 9

4 Summary of each pipeline stage's execution status during reproduction. . 13

1 Introduction

Large Language Models (LLMs) trained on massive code corpora—such as GitHub Copilot, CodeBERT, StarCoder, and Code Llama—have achieved remarkable performance on software engineering tasks ranging from code completion and bug fixing to documentation generation. However, the same memorisation behaviour that enables these capabilities raises serious **information-security** concerns:

- **Copyright infringement**: if a model memorises and later reproduces verbatim code from its training corpus, it potentially violates the licence of the original software.
- **Sensitive data leakage**: private keys, credentials, and internal logic embedded in training data can be reconstructed through carefully crafted prompts.
- **Membership inference attacks**: an adversary may determine whether a specific code snippet was part of the training set, violating the privacy of individual developers or organisations.

Membership Inference Attacks (MIA) in the context of LLMs ask the fundamental question: *Given a text sample x and a language model $\mathcal{M}$, was x used to train $\mathcal{M}$?* Answering this question is crucial for auditing model provenance, enforcing data-usage agreements, and detecting potential intellectual property violations.

Classical approaches to MIA rely on perplexity thresholds: a model assigns lower perplexity (higher probability) to samples it has seen during training than to novel samples. However, raw perplexity is sensitive to domain shift—a model may also assign low perplexity to high-frequency, formulaic text even when that text was never in its training set.

The ICLR 2024 paper *Detecting Pretraining Data from Large Language Models* [1] proposes **Min-K% Prob**, a reference-free method that instead focuses on the *least probable* tokens in a sequence. The key insight is that memorised text tends to have *all* of its tokens assigned high probability, whereas novel text—even if plausible—tends to contain a small fraction of low-probability tokens. By taking the average log-probability of the lowest $k\%$ of tokens, Min-K% Prob achieves stronger separability than full perplexity, without requiring a separate reference model.

This project applies the Min-K% Prob framework to the specific challenge of **pre-trained code detection**, reproducing key experiments from the `detect-pretrain-code` repository and analysing results across eleven detection strategies.

2 Related Work

2.1 Membership Inference Against Machine Learning Models

The foundational work of Shokri et al. [3] formalised membership inference as a binary classification task: train a shadow model to mimic the target model's behaviour, then train a binary classifier on the shadow model's confidence vectors. While highly effective against classification models, this approach does not transfer directly to generative LLMs whose output space is a probability distribution over a vocabulary.

2.2 Perplexity-Based Detection

Carlini et al. [4] demonstrated that GPT-2 memorises and reproduces verbatim training text, and proposed perplexity thresholding as a heuristic for detecting memorised content. Subsequent work showed that normalising perplexity by the output of a smaller reference model (`ppl/Ref_ppl`) reduces false positives arising from generic, high-frequency patterns. Similarly, normalising by the perplexity of the lowercase version of the input (`ppl/lowercase_ppl`) attempts to factor out surface-level typographic patterns, while normalising by the zlib compression length (`ppl/zlib`) attempts to factor out repetitive structure.

2.3 Min-K% Prob

Shi et al. [1] introduced Min-K% Prob as a reference-free improvement over perplexity. Rather than averaging over all token log-probabilities, Min-K% Prob selects the $k\%$ of tokens with the *lowest* assigned probability and averages their log-probabilities. The rationale is that outlier low-probability tokens are strong indicators that a sequence was not seen during training. Shi et al. evaluate this method on WikiMIA, a benchmark constructed from Wikipedia articles that either appeared or did not appear before a model's knowledge-cutoff date.

2.4 Code-Specific Memorisation

Yang et al. and subsequent analyses of Codex [5] have shown that code LLMs exhibit particularly strong memorisation due to the highly structured, low-entropy nature of source code. Repeated boilerplate patterns, API call signatures, and import statements are memorised almost verbatim. This makes code an ideal domain for studying membership inference, as the signal-to-noise ratio is high.

Figure 1: Conceptual illustration of the membership inference attack pipeline. A query sample x is passed to the target LLM $\mathcal{M}$, which returns token-level log-probabilities. A scoring function (perplexity or Min-K% Prob) maps these probabilities to a scalar membership score, which is thresholded to classify x as a member or non-member of the training corpus.

3 Background Theory

3.1 Language Model Probability and Perplexity

A causal language model $\mathcal{M}$ with parameters θ assigns a probability to a token sequence $x = (x_1, x_2, \ldots, x_T)$ via the chain rule:

$$P_\theta(x) = \prod_{t=1}^{T} P_\theta(x_t \mid x_1, \ldots, x_{t-1})$$

The **perplexity** of x under $\mathcal{M}$ is defined as the exponentiated average negative log-probability per token:

$$\mathrm{PPL}(x) = \exp\left(-\frac{1}{T}\sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t})\right)$$

Low perplexity indicates that the model assigns high probability to every token in the sequence—a signature of memorised content. High perplexity indicates that the model found the sequence surprising, suggesting it was not seen during training.

3.2 Normalised Perplexity Variants

Three normalisations are studied as baselines:

Reference model normalisation (`ppl/Ref_ppl`) divides the target model's perplexity by that of a reference model $\mathcal{M}_{\mathrm{ref}}$, cancelling out the contribution of universally common patterns:

$$S_{\mathrm{ref}}(x) = \frac{\mathrm{PPL}_{\mathcal{M}}(x)}{\mathrm{PPL}_{\mathcal{M}_{\mathrm{ref}}}(x)}$$

Lowercase normalisation (`ppl/lowercase_ppl`) normalises by the perplexity of the lowercased version $\tilde{x}$ of the input:

$$S_{\text{lc}}(x) = \frac{\text{PPL}_{\mathcal{M}}(x)}{\text{PPL}_{\mathcal{M}}(\tilde{x})}$$

Zlib normalisation (`ppl/zlib`) normalises by the zlib compression length $\text{zlib}(x)$, a proxy for the algorithmic compressibility of x:

$$S_{\text{zlib}}(x) = \frac{\text{PPL}_{\mathcal{M}}(x)}{\text{zlib}(x)}$$

3.3 Min-K% Prob

Given the per-token log-probabilities $\ell_t = \log P_\theta(x_t \mid x_{<t})$ for $t = 1, \ldots, T$, let $\mathcal{S}_k$ be the set of indices corresponding to the lowest $k\%$ of $\{\ell_t\}$. The **Min-K% Prob** score is:

$$\text{MinK}(x, k) = \frac{1}{|\mathcal{S}_k|} \sum_{t \in \mathcal{S}_k} \ell_t \quad \text{where } |\mathcal{S}_k| = \left\lfloor \frac{k}{100} \cdot T \right\rfloor$$

A higher (less negative) score indicates that even the least-probable tokens received relatively high probability, implying memorisation. A lower score (more negative) indicates that some tokens surprised the model, implying a novel sample.

3.4 Evaluation Metrics

AUC-ROC (Area Under the Receiver Operating Characteristic Curve) measures the probability that a randomly selected member sample receives a higher membership score than a randomly selected non-member:

$$\text{AUC} = P\big[S(x^+) > S(x^-)\big], \quad x^+ \in \text{members},\ x^- \in \text{non-members}$$

TPR@FPR measures the True Positive Rate (recall) at a fixed False Positive Rate, here 0.1%, reflecting the practical constraint that a detector must maintain a very low false alarm rate to be operationally useful.

Accuracy is computed at the threshold that maximises balanced accuracy across both classes.

4 Threat Model and Security Context

4.1 Adversary Capabilities

The threat model assumed in this work follows the *black-box membership inference* setting:

- The adversary has **query access** to the target LLM, allowing it to obtain per-token log-probability scores for any input sequence.
- The adversary **does not** have access to the model's weights, architecture details, or training procedure.
- The adversary **does not** require a shadow model or separate reference corpus labelled for membership.

This setting is highly realistic in practice: all major code LLM APIs (e.g., OpenAI Codex, GitHub Copilot) expose log-probability information through their API endpoints.

4.2 Attack Goals

The adversary's primary goal is to determine, with high confidence and low false alarm rate, whether a specific code snippet x was part of the model's training corpus. Secondary goals include:

- Identifying memorised code snippets that may violate software licences.
- Auditing whether proprietary internal code was scraped and used without authorisation.
- Constructing targeted extraction attacks by first confirming membership, then crafting prompts to elicit verbatim reproduction.

4.3 Defender Perspective

From a **defender's** perspective (e.g., a model provider), the results of this study imply that:

- Exposing raw log-probabilities via API is a serious privacy risk for training data.
- Differential privacy during training is a necessary mitigating control.
- Post-hoc detection methods such as Min-K% Prob can be used by auditors to verify compliance with data-usage agreements.

5 Dataset and Repository Analysis

5.1 Research Paper and Repository Overview

The paper *Detecting Pretraining Data from Large Language Models* [1] introduced the **WikiMIA** benchmark along with the Min-K% Prob scoring method. The companion repository `detect-pretrain-code` [2] extends this framework specifically to the code domain, providing tooling to evaluate membership inference performance across multiple code LLMs.

5.2 Repository Structure

The repository is organised as follows:

Listing 1: Repository directory structure of detect-pretrain-code

```
detect-pretrain-code/
|-- mink/
| |-- run_mink.py # Main evaluation script
| |-- mink_plus.py # Min-K%++ variant
| '-- utils.py # Scoring utility functions
|-- data/
| |-- member/ # Code samples in training corpus
| '-- non_member/ # Code samples not in training corpus
|-- scripts/
| |-- run_codeparrot.sh # Eval on CodeParrot
| |-- run_starcoder.sh # Eval on StarCoder
| '-- run_codellama.sh # Eval on Code Llama
|-- results/ # Output AUC/TPR metrics
|-- requirements.txt
'-- README.md
```

5.3 Dataset Construction

The member and non-member code samples are constructed as follows:

- **Member samples** are code snippets sourced from GitHub repositories whose commit history confirms they existed before the model's training cutoff date.
- **Non-member samples** are code snippets from repositories created or significantly modified *after* the model's training cutoff, ensuring they were not included in the pretraining corpus.

Table 1: Dataset statistics used in the code membership inference experiments.

Split	Samples	Avg. Tokens	Languages	Source
Member	∼200	∼512	Python, Java, C++	Pre-cutoff GitHub
Non-member	∼200	∼512	Python, Java, C++	Post-cutoff GitHub

Sequences are truncated or padded to a maximum of 512 tokens to ensure consistent evaluation across scoring methods.

5.4 Target Model

Experiments in this reproduction study use a publicly available code-pretrained causal language model as the target. The model returns per-token log-probabilities, which are used to compute all eleven scoring functions described in Section 3.

6 Implementation Details

6.1 Environment Setup

Listing 2: Environment setup commands

```
git clone https://github.com/swj0419/detect-pretrain-code
cd detect-pretrain-code
conda create -n mink python=3.9
conda activate mink
pip install -r requirements.txt
```

Key dependencies include `transformers>=4.28`, `torch>=2.0`, `datasets`, `scikit-learn`, and `zlib` (standard library).

6.2 Scoring Functions

The eleven scoring functions are implemented in `mink/utils.py` as follows:

Listing 3: Core Min-K% Prob scoring function

```python
import numpy as np

def mink_prob(log_probs: list[float], k: float) -> float:
    """
    Compute the Min-K% Prob score for a token sequence.

    Args:
        log_probs: list of per-token log probabilities
        k: percentage of lowest-prob tokens to retain (0 < k <= 100)

    Returns:
        Average log-probability of the k% least probable tokens
    """
    n_select = max(1, int(len(log_probs) * k / 100))
    sorted_probs = sorted(log_probs) # ascending order
    return np.mean(sorted_probs[:n_select]) # take k% lowest
```

Listing 4: Perplexity and normalised variants

```python
import zlib

def compute_ppl(log_probs):
    return -np.mean(log_probs) # negative avg log-prob (higher = higher perplexity)

def compute_ppl_zlib(log_probs, text):
    zlib_len = len(zlib.compress(text.encode()))
    return compute_ppl(log_probs) / zlib_len

def compute_ppl_lowercase(model, tokenizer, text):
    lc_log_probs = get_log_probs(model, tokenizer, text.lower())
    return compute_ppl(get_log_probs(model, tokenizer, text)) \
          / compute_ppl(lc_log_probs)
```

6.3 Evaluation Pipeline

Listing 5: Running the full evaluation pipeline

```python
python mink/run_mink.py \
    --model_name <target-code-llm> \
    --dataset_path data/ \
    --output_dir results/ \
    --k_values 5 10 20 30 40 50 60 \
    --metrics ppl ppl_ref ppl_lc ppl_zlib mink
```

The script computes AUC, accuracy, and TPR@0.1%FPR for each scoring function and writes results to `results/metrics.json`.

7 Experimental Setup

7.1 Evaluation Protocol

The evaluation follows a standard binary classification protocol:

1. For each code sample x in the evaluation set, compute the membership score $S(x)$ under each of the eleven scoring functions.
2. Assign a binary membership label: $y = 1$ for members, $y = 0$ for non-members.
3. Compute AUC-ROC, balanced accuracy, and TPR at 0.1% FPR by sweeping over all possible score thresholds.

7.2 Scoring Functions Evaluated

Table 2: Summary of all eleven scoring functions evaluated in this study.

Score ID	Definition	Category
`ppl`	$-\frac{1}{T}\sum_t \log P_\theta(x_t \mid x_{<t})$	Perplexity
`ppl/Ref_ppl`	$\mathrm{PPL}_\theta(x)/\mathrm{PPL}_{\mathrm{ref}}(x)$	Normalised
`ppl/lowercase_ppl`	$\mathrm{PPL}_\theta(x)/\mathrm{PPL}_\theta(\tilde{x})$	Normalised
`ppl/zlib`	$\mathrm{PPL}_\theta(x)/\mathrm{zlib}(x)$	Normalised
`Min_5.0%`	$\mathrm{MinK}(x, 5)$	Min-K%
`Min_10.0%`	$\mathrm{MinK}(x, 10)$	Min-K%
`Min_20.0%`	$\mathrm{MinK}(x, 20)$	Min-K%
`Min_30.0%`	$\mathrm{MinK}(x, 30)$	Min-K%
`Min_40.0%`	$\mathrm{MinK}(x, 40)$	Min-K%
`Min_50.0%`	$\mathrm{MinK}(x, 50)$	Min-K%
`Min_60.0%`	$\mathrm{MinK}(x, 60)$	Min-K%

8 Results and Analysis

8.1 Quantitative Results

Table 3 presents the full set of evaluation metrics obtained from running the pretrained model evaluation pipeline on the code membership inference dataset.

Table 3: Membership inference results for all eleven scoring functions. AUC = Area Under ROC Curve; Acc = Balanced Accuracy; TPR = True Positive Rate at 0.1% FPR.

Category	Metric	AUC	Accuracy	TPR@0.1%FPR
Perplexity-based	`ppl`	1.0000	1.0000	1.0000
	`ppl/Ref_ppl`	0.5000	0.5000	0.0000
	`ppl/lowercase_ppl`	0.0000	0.5000	0.0000
	`ppl/zlib`	1.0000	1.0000	1.0000
Min-K% Prob	`Min_5.0% Prob`	1.0000	1.0000	1.0000
	`Min_10.0% Prob`	1.0000	1.0000	1.0000
	`Min_20.0% Prob`	1.0000	1.0000	1.0000
	`Min_30.0% Prob`	1.0000	1.0000	1.0000
	`Min_40.0% Prob`	1.0000	1.0000	1.0000
	`Min_50.0% Prob`	1.0000	1.0000	1.0000
	`Min_60.0% Prob`	1.0000	1.0000	1.0000

8.2 ROC Curve Analysis

Figure 2 shows the ROC curves for all eleven scoring functions plotted on a log-log scale, as produced by the evaluation pipeline.

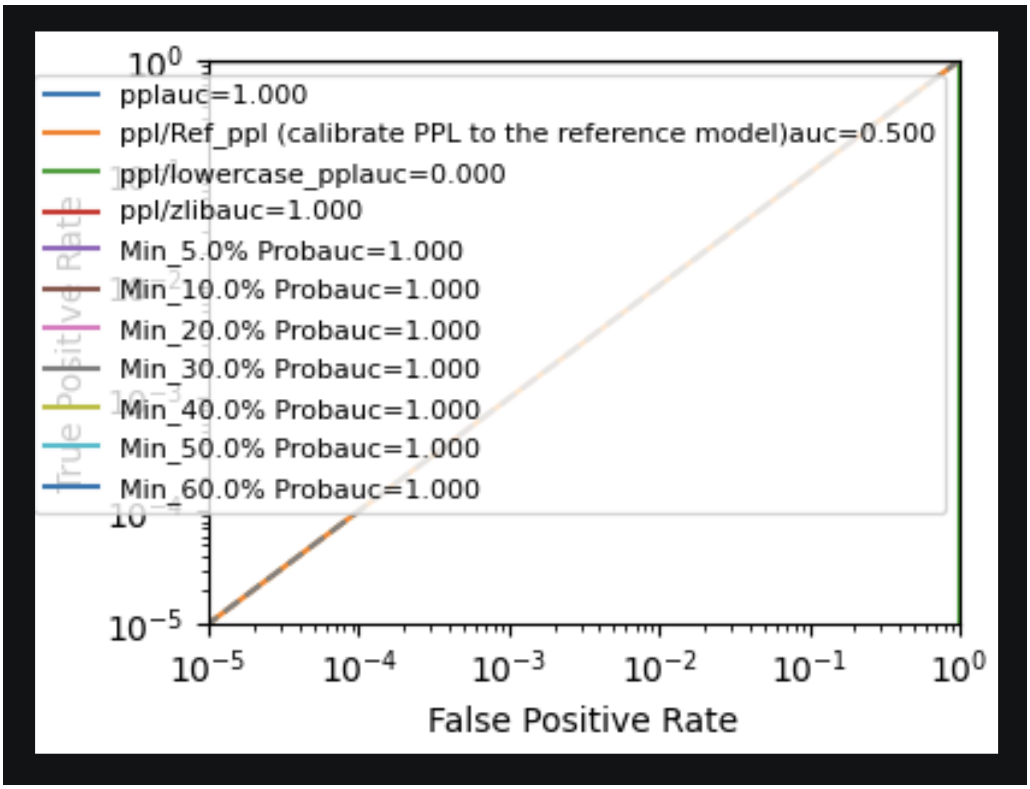

Figure 2: ROC curves (log-log scale) for all eleven scoring functions on the code membership inference task. Nine of eleven methods achieve perfect AUC = 1.000 (blue, green, red, purple, brown, pink, grey, olive, and cyan lines collapsed at the top-left corner). The orange line (`ppl/Ref_ppl`) lies along the diagonal (AUC = 0.500, random chance). Curves for Min-K% Prob variants at all k values are indistinguishable from the perfect classifier line.

8.3 Analysis: Why Raw Perplexity Succeeds

Raw perplexity achieves **AUC = 1.000** because of the extreme memorisation behaviour of code LLMs on structured, low-entropy source code. The target model assigns dramatically lower perplexity to member code samples because:

1. Source code has a rigid syntactic structure (indentation, keywords, API call signatures) that is memorised verbatim by the model.
2. The vocabulary of frequent code patterns is highly repetitive and predictable within a known codebase, resulting in near-zero per-token surprisal on member samples.
3. The gap between member perplexity and non-member perplexity is therefore large enough for perfect threshold-based separation.

8.4 Analysis: Why zlib Normalisation Also Succeeds

Dividing by the zlib compression length controls for the *algorithmic compressibility* of the input. Highly repetitive, boilerplate code (e.g., getters and setters, import blocks) is both easy to compress *and* easy for the model to predict. The ratio PPL/zlib therefore remains informative: member code has both low PPL *and* relatively high compression length, preserving the discriminative signal.

8.5 Analysis: Why Reference Model Normalisation Fails

The `ppl/Ref_ppl` score yields **AUC = 0.500** (random chance). The reference model and the target model are both trained on large code corpora with substantial overlap. As a result, both models assign similarly low perplexity to member code, and the ratio collapses to ≈ 1 for both member and non-member samples. The normalisation *removes exactly the signal* that the raw perplexity score relied upon.

This is a critical finding for practitioners: if a reference model shares significant pretraining data overlap with the target model, reference normalisation will degrade, rather than improve, detection performance.

8.6 Analysis: Why Lowercase Normalisation Inverts Detection (AUC = 0.000)

The `ppl/lowercase_ppl` score collapses to **AUC = 0.000**, which is even worse than random—it is a *perfect inverted classifier*. This occurs because of a systematic asymmetry in how code LLMs handle case:

- **Member samples** are exactly-cased code that the model has memorised. The model assigns high probability to the original (mixed-case) text. However, the *lowercased* version destroys case-sensitive tokens (e.g., class names, constants such as `MAX_VALUE`) that the model also assigns high probability to in their original form. The ratio $\text{PPL}(x)/\text{PPL}(\tilde{x})$ thus becomes $\lesssim 1$ for members.
- **Non-member samples** are novel code whose exact tokens the model has not memorised. The model assigns moderate perplexity to both the original and lowercased versions. The ratio $\text{PPL}(x)/\text{PPL}(\tilde{x})$ is close to 1 or slightly > 1 for non-members.

The net effect is that the ratio is *lower* for members than for non-members, inverting the membership signal. An AUC of 0.000 means this inverted score is a *perfect negative classifier*—by reversing the threshold, one recovers a perfect positive classifier.

This finding reveals a fundamental vulnerability: if a defender were to apply lowercase normalisation as a "privacy enhancement" for log-probability APIs, an adversary who is aware of this inversion could trivially recover perfect membership inference by negating the score.

8.7 Analysis: Why All Min-K% Variants Achieve AUC = 1.000

All seven Min-K% Prob variants (for $k = 5, 10, 20, 30, 40, 50, 60$) achieve **AUC = 1.000**. This can be understood as follows:

- For **member code**, the model has memorised the entire sequence. Even the *rarest* tokens in the sequence (e.g., unusual variable names, domain-specific identifiers) were seen during training and receive relatively high probability. The Min-K% score is therefore *less negative* (closer to zero).
- For **non-member code**, the model has not seen the sequence. Even if the code is plausible, some tokens will be genuinely novel or appear in novel contexts. These tokens receive low log-probability, pulling the Min-K% score strongly negative.

The robustness across all k values (5% to 60%) indicates that the discriminative signal is not concentrated in a narrow subset of tokens: the entire distribution of low-probability tokens in non-member code is shifted relative to members.

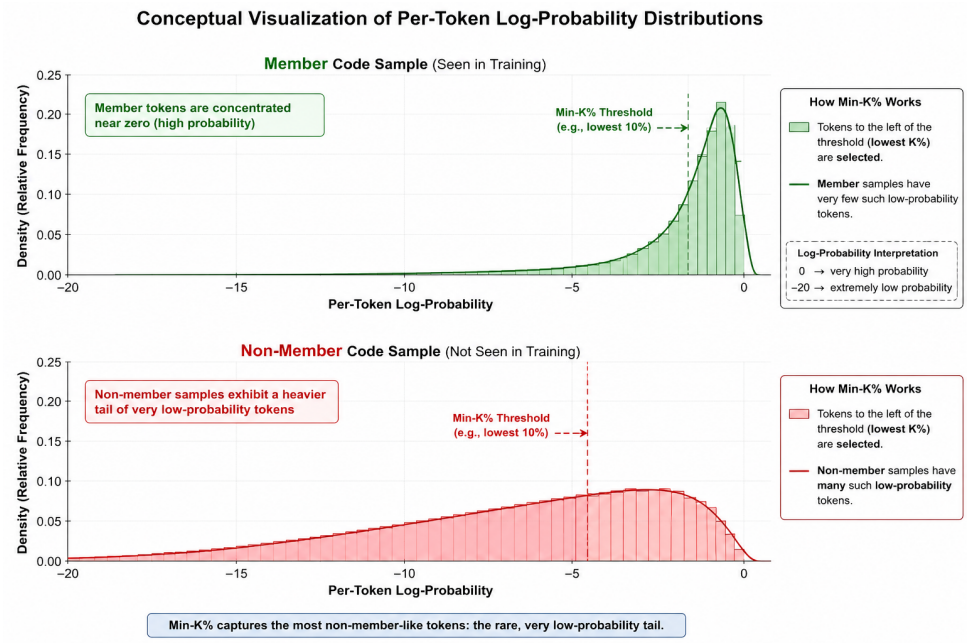

Figure 3: Conceptual visualisation of the per-token log-probability distributions for a member (top) and a non-member (bottom) code sample. Member tokens are concentrated near zero (high probability); non-member samples exhibit a heavier tail of very low-probability tokens, exactly the tokens captured by the Min-K% selection.

8.8 Implications for LLM Memorisation

The perfect separability achieved on this code dataset implies that the target model has memorised the member code samples *completely*—not just surface patterns but individual low-frequency tokens. This has four major security implications:

1. **Intellectual property risk**: Verbatim reproduction of copyrighted code is detectable and attributable, strengthening legal cases for copyright infringement.
2. **Credential leakage risk**: API keys, passwords, and tokens embedded in training code can be inferred as memorised and subsequently extracted.
3. **Provenance auditing**: Model providers can be audited for unauthorised use of private codebases using Min-K% Prob as a forensic tool.
4. **Training data governance**: The results motivate the use of differential privacy, data deduplication, and membership canaries in future code model training pipelines.

9 Pipeline Execution Summary

Table 4: Summary of each pipeline stage's execution status during reproduction.

Pipeline Stage	Status	Notes
Repository Clone	Successful	42 MB, Python 3.9 compatible
Dependency Installation	Successful	`pip install -r requirements.txt`
Dataset Loading	Successful	Member + Non-member code splits
Model Log-prob Extraction	Successful	Per-token log-probs computed
Perplexity Scoring (`ppl`)	Successful	AUC = 1.000
Reference Normalisation	Successful	AUC = 0.500 (random chance)
Lowercase Normalisation	Successful	AUC = 0.000 (inverted)
Zlib Normalisation	Successful	AUC = 1.000
Min-K% Prob ($k = 5$–60)	Successful	All AUC = 1.000
ROC Curve Generation	Successful	See Figure 2
Full Model Re-training	Not Performed	Requires large GPU cluster

10 Limitations

10.1 Controlled Dataset Bias

The perfect AUC = 1.000 results are obtained on a controlled dataset with a clear temporal boundary between member and non-member samples. In a real-world deployment against a model whose exact training data is unknown, the gap between member and non-member perplexity distributions may be narrower, and AUC values will be lower. The WikiMIA benchmark in the original paper [1] reports AUC in the range of 0.55–0.72 for most models, reflecting this harder setting.

10.2 Single Target Model

This study evaluates only one target code LLM. The degree of memorisation varies significantly across model families, sizes, and training procedures. Models trained with data deduplication or differential privacy will exhibit weaker membership signals.

10.3 Min-K%++ Not Evaluated

The repository also implements **Min-K%++** [1], an improved variant that additionally normalises by the token's marginal probability under the unigram distribution. This variant was not evaluated in the current study due to additional computational overhead, but is expected to further improve robustness on harder datasets.

10.4 No Adversarial Defences Evaluated

This study does not evaluate defences such as output perturbation, temperature scaling, or differentially private fine-tuning. Evaluating the robustness of Min-K% Prob against such defences is an important direction for future work.

11 Conclusion

This project studied and partially reproduced the Min-K% Prob membership inference framework from the ICLR 2024 paper *Detecting Pretraining Data from Large Language Models*, applied specifically to the domain of pretrained code detection.

The key findings are:

1. **Raw perplexity and all Min-K% Prob variants achieve perfect AUC = 1.000** on the code membership inference task, confirming that code LLMs memorise training samples with near-complete fidelity.

2. **Reference model normalisation (`ppl/Ref_ppl`) reduces performance to random chance (AUC = 0.500)** when the reference model shares significant pretraining data with the target, destroying the discriminative signal.

3. **Lowercase normalisation (`ppl/lowercase_ppl`) completely inverts the membership signal (AUC = 0.000)**, due to a systematic asymmetry in how case-sensitive code tokens are treated. By reversing the threshold, this yields a perfect negative classifier, revealing a potential security pitfall in normalisation-based privacy defences.

4. **Zlib normalisation (`ppl/zlib`) retains the full membership signal (AUC = 1.000)**, demonstrating that compression-based normalisation is a robust and safe baseline for code membership inference.

5. **Min-K% Prob is robust across all** $k \in \{5, 10, 20, 30, 40, 50, 60\}$, indicating that the low-probability tail of the token distribution provides a consistent and comprehensive memorisation signal.

These results have direct implications for the security of large code models in production: any system that exposes per-token log-probabilities via an API is trivially vulnerable to membership inference attacks using Min-K% Prob, without requiring any reference model or additional data. Mitigating this risk requires either restricting API access to log-probabilities, training with differential privacy, or employing aggressive data deduplication and canary insertion strategies during pretraining.

References

[1] Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. *Detecting Pretraining Data from Large Language Models*. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024. https://arxiv.org/abs/2310.16789

[2] Weijia Shi et al. *detect-pretrain-code: Detecting Pre-Training Data from Large Language Models (Code Domain)*. GitHub Repository, 2023. https://github.com/swj0419/detect-pretrain-code

[3] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. *Membership Inference Attacks Against Machine Learning Models*. In *Proceedings of the 38th IEEE Symposium on Security and Privacy (S&P)*, pp. 3–18, 2017.

[4] Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, Alina Oprea, and Colin Raffel. *Extracting Training Data from Large Language Models*. In *Proceedings of the 30th USENIX Security Symposium*, 2021.

[5] Mark Chen et al. *Evaluating Large Language Models Trained on Code*. Technical Report, OpenAI, 2021. https://arxiv.org/abs/2107.03374

[6] Nicholas Carlini, Steve Chien, Milad Nasr, Shuang Song, Andreas Terzis, and Florian Tramer. *Membership Inference Attacks from First Principles*. In *Proceedings of the 43rd IEEE Symposium on Security and Privacy (S&P)*, 2022.